

✓ Lab 10 – La Voix du Consommateur

Analyse de sentiment & NLP sur les avis Amazon

```
# — 0. Installation des dépendances —————
!pip install textblob wordcloud nltk matplotlib --quiet
!python -m textblob.download_corpora

import nltk
for pkg in ['punkt', 'stopwords', 'averaged_perceptron_tagger',
            'wordnet', 'omw-1.4', 'punkt_tab',
            'averaged_perceptron_tagger_eng']:
    nltk.download(pkg, quiet=True)

print('Setup terminé ✓')
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data] Unzipping corpora/brown.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[nltk_data] Downloading package conll2000 to /root/nltk_data...
[nltk_data] Unzipping corpora/conll2000.zip.
[nltk_data] Downloading package movie_reviews to /root/nltk_data...
[nltk_data] Unzipping corpora/movie_reviews.zip.
Finished.
Setup terminé ✓
```

```
# — 1. Chargement du dataset —————
import pandas as pd

# Option A - dataset depuis le dépôt du cours :
# from google.colab import files
# uploaded = files.upload() # uploadez amazon_review.csv
# df = pd.read_csv(list(uploaded.keys())[0])

# Option B - dataset public Amazon Reviews (Kaggle mirror sur GitHub)
url = 'https://raw.githubusercontent.com/pycaret/pycaret/master/datasets/amazon.csv'
df = pd.read_csv(url)

# Normaliser le nom des colonnes (adapter si nécessaire)
print('Colonnes disponibles :', df.columns.tolist())
print(df.shape)
df.head()
```

Colonnes disponibles : ['reviewText', 'Positive']
(20000, 2)

	reviewText	Positive	
0	This is a one of the best apps according to a b...	1	
1	This is a pretty good version of the game for ...	1	
2	this is a really cool game. there are a bunch ...	1	
3	This is a silly game and can be frustrating, b...	1	
4	This is a terrific game on any pad. Hrs of fun...	1	

Étapes suivantes : [Générer du code avec df](#) [New interactive sheet](#)

```
# — Adaptation : on s'assure d'avoir une colonne 'reviewText' —————
# Renommer si nécessaire selon le dataset utilisé
if 'reviewText' not in df.columns:
    # Cherche une colonne texte probable
    text_col = [c for c in df.columns if 'text' in c.lower() or 'review' in c.lower()]
    if text_col:
        df = df.rename(columns={text_col[0]: 'reviewText'})
    else:
        raise ValueError('Aucune colonne texte trouvée. Vérifiez votre CSV.')

# On garde uniquement les lignes non-vides
df = df[['reviewText']].dropna().reset_index(drop=True)
# Pour alléger sur Colab, limiter à 5000 avis si le dataset est très grand
if len(df) > 5000:
    df = df.sample(5000, random_state=42).reset_index(drop=True)
```

```
print(f'{len(df)} avis chargés')
df.head(3)
```

5000 avis chargés

reviewText 

- | | reviewText |
|---|---|
| 0 | this app is great, especially when i get stuck... |
| 1 | Doesn't work that great on my Kindle fire. Wha... |
| 2 | Was a bit hard to find out how to search. But ... |

Étapes suivantes : [Générer du code avec df](#) [New interactive sheet](#)

✓ Tâche 1 – Analyse de sentiment & bar chart

```
from textblob import TextBlob
import matplotlib.pyplot as plt

def classify_sentiment(text):
    polarity = TextBlob(str(text)).sentiment.polarity
    if polarity > 0.1:
        return 'positif'
    elif polarity < -0.1:
        return 'négatif'
    else:
        return 'neutre'

df['sentiment'] = df['reviewText'].apply(classify_sentiment)
df['polarity'] = df['reviewText'].apply(lambda t: TextBlob(str(t)).sentiment.polarity)

sentiment_counts = df['sentiment'].value_counts()

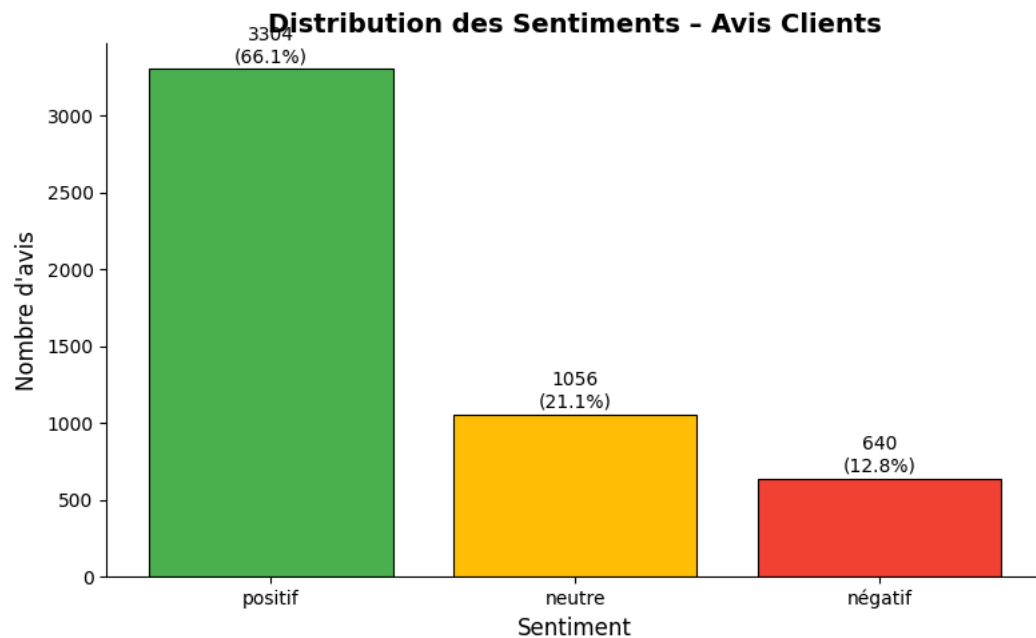
colors = {'positif': '#4CAF50', 'neutre': '#FFC107', 'négatif': '#F44336'}
bar_colors = [colors.get(s, 'steelblue') for s in sentiment_counts.index]

fig, ax = plt.subplots(figsize=(8, 5))
bars = ax.bar(sentiment_counts.index, sentiment_counts.values,
              color=bar_colors, edgecolor='black', linewidth=0.8)

# Annotations
for bar, val in zip(bars, sentiment_counts.values):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 20,
            f'{val}\n({val/len(df)*100:.1f}%)',
            ha='center', va='bottom', fontsize=10)

ax.set_title('Distribution des Sentiments - Avis Clients', fontsize=14, fontweight='bold')
ax.set_xlabel('Sentiment', fontsize=12)
ax.set_ylabel('Nombre d\'avis', fontsize=12)
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.savefig('sentiment_distribution.png', dpi=150)
plt.show()

print(sentiment_counts)
```



```
sentiment
positif    3304
neutre     1056
négatif     640
Name: count, dtype: int64
```

✓ Tâche 2 – Prétraitement & Lemmatisation

```
import re
import string
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
from nltk import pos_tag
from nltk.corpus import wordnet

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def get_wordnet_pos(treebank_tag):
    """Convertit les POS tags NLTK en tags WordNet."""
    if treebank_tag.startswith('J'):
        return wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return wordnet.VERB
    elif treebank_tag.startswith('N'):
        return wordnet.NOUN
    elif treebank_tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN # défaut

def preprocess(text):
    # 1. Minuscules
    text = str(text).lower()
    # 2. Supprimer ponctuation et chiffres
    text = re.sub(r'^a-z\s]', '', text)
    # 3. Tokenization
    tokens = word_tokenize(text)
    # 4. Supprimer stopwords et tokens courts
    tokens = [t for t in tokens if t not in stop_words and len(t) > 2]
    # 5. POS tagging + Lemmatisation
    pos_tags = pos_tag(tokens)
    lemmas = [lemmatizer.lemmatize(word, get_wordnet_pos(tag))
               for word, tag in pos_tags]
    return lemmas

print('Prétraitement en cours...')
df['tokens'] = df['reviewText'].apply(preprocess)
df['pos_tags'] = df['reviewText'].apply(
    lambda t: pos_tag([w for w in word_tokenize(str(t).lower())
                       if w not in stop_words and w.isalpha() and len(w) > 2])
)
```

```
# Exemple de résultat
print('\nExemple de review prétraitée :')
print('Original  :', df['reviewText'][0][:120])
print('Lemmatisé :', df['tokens'][0][:15])
```

Prétraitement en cours...

Exemple de review prétraitée :

Original : this app is great, especially when i get stuck with all vowels and one letter, it helps me find words i can use
Lemmatisé : ['app', 'great', 'especiall', 'get', 'stuck', 'vowel', 'one', 'letter', 'help', 'find', 'word', 'use', 'word',

✓ Tâche 3 – Top 15 des mots les plus fréquents

```
from collections import Counter

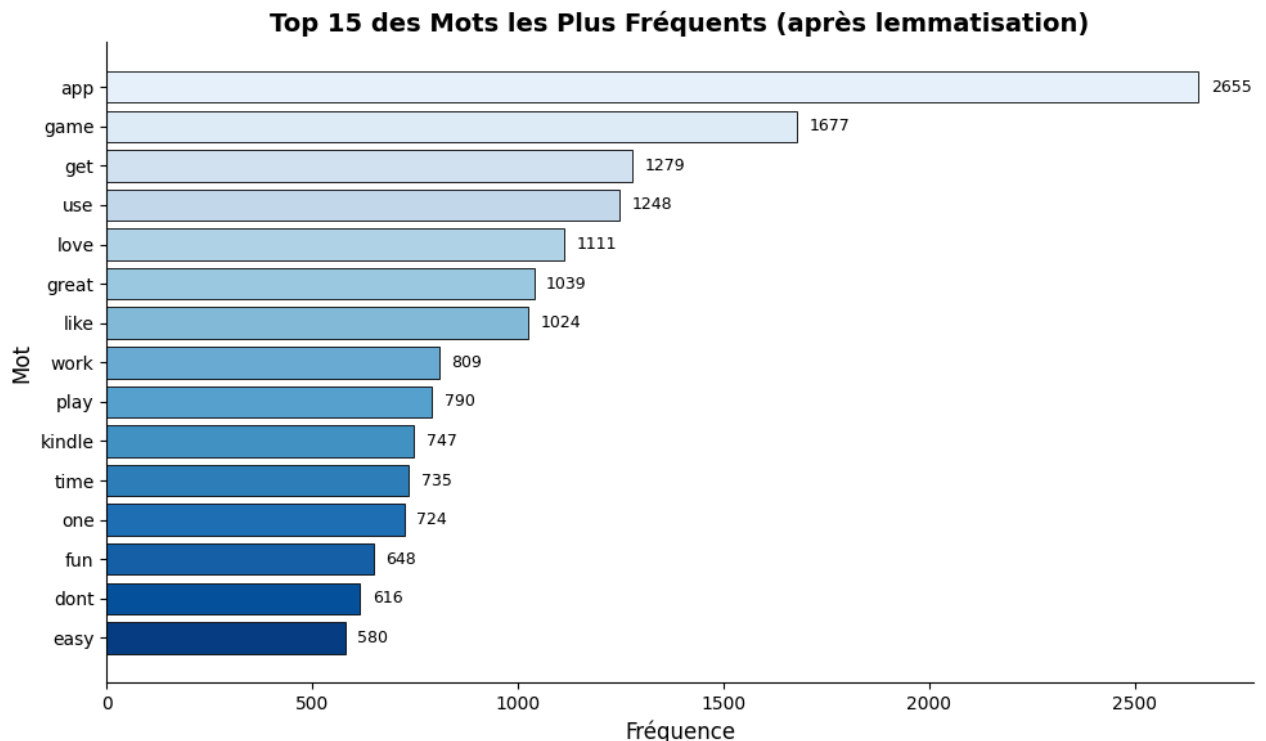
# Corpus complet de lemmes
all_tokens = [token for tokens in df['tokens'] for token in tokens]
freq = Counter(all_tokens)
top15 = freq.most_common(15)

words_top, counts_top = zip(*top15)

fig, ax = plt.subplots(figsize=(10, 6))
bars = ax.barh(words_top[:-1], counts_top[:-1],
               color=plt.cm.Blues_r(range(15, 255, 16)),
               edgecolor='black', linewidth=0.6)

for bar, count in zip(bars, counts_top[:-1]):
    ax.text(bar.get_width() + 30, bar.get_y() + bar.get_height() / 2,
            str(count), va='center', fontsize=9)

ax.set_title('Top 15 des Mots les Plus Fréquents (après lemmatisation)', fontsize=14, fontweight='bold')
ax.set_xlabel('Fréquence', fontsize=12)
ax.set_ylabel('Mot', fontsize=12)
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.savefig('top15_words.png', dpi=150)
plt.show()
```



✓ Tâche 4 – Nuage de mots (Word Cloud)

```
from wordcloud import WordCloud

corpus_text = ' '.join(all_tokens)
```



```

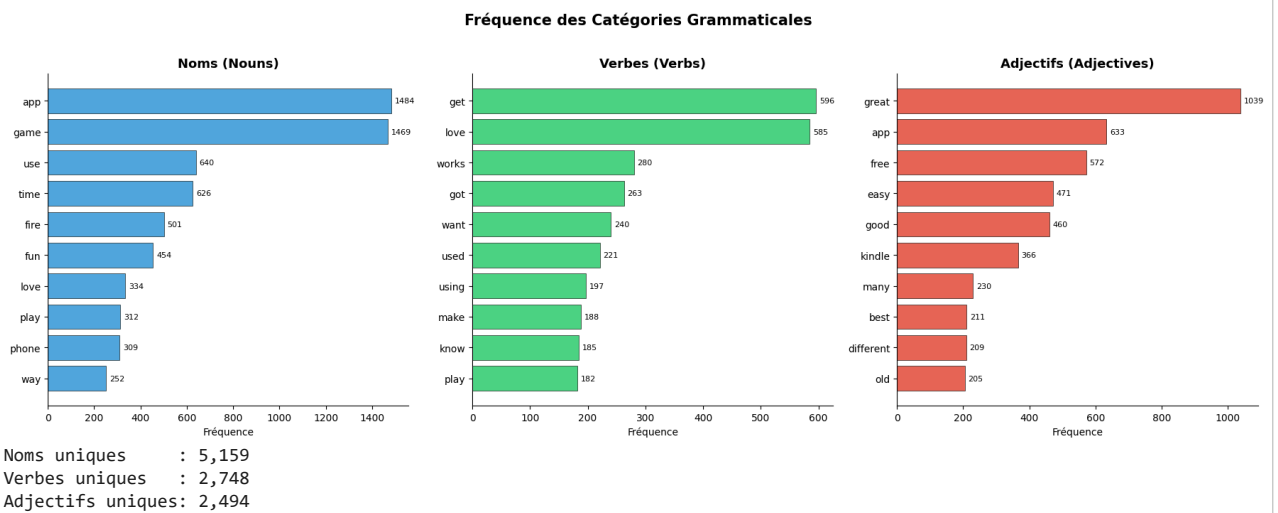
(top_verbs, 'Verbes (Verbs)', '#2ECC71'),
(top_adjs, 'Adjectifs (Adjectives)', '#E74C3C'),
]

for ax, (data, title, color) in zip(axes, datasets):
    if not data:
        ax.text(0.5, 0.5, 'Aucune donnée', ha='center', va='center')
        ax.set_title(title)
        continue
    words_list, counts_list = zip(*data)
    bars = ax.barh(words_list[::-1], counts_list[::-1],
                   color=color, alpha=0.85, edgecolor='black', linewidth=0.5)
    for bar, count in zip(bars, counts_list[::-1]):
        ax.text(bar.get_width() + max(counts_list) * 0.01,
                bar.get_y() + bar.get_height() / 2,
                str(count), va='center', fontsize=8)
    ax.set_title(title, fontsize=13, fontweight='bold')
    ax.set_xlabel('Fréquence', fontsize=10)
    ax.spines[['top', 'right']].set_visible(False)

fig.suptitle('Fréquence des Catégories Grammaticales',
             fontsize=15, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('pos_frequency.png', dpi=150, bbox_inches='tight')
plt.show()

print(f'Noms uniques      : {len(set(nouns))},}')
print(f'Verbes uniques     : {len(set(verbs))},}')
print(f'Adjectifs uniques: {len(set(adjectives))},}')

```



Résumé

Tâche	Description	Sortie
1	Analyse de sentiment (TextBlob)	Bar chart
2	Prétraitement + lemmatisation POS	Colonne <code>tokens</code>
3	Top 15 mots fréquents	Bar chart horizontal